

PRACTICAL EXPERIMENT – 5

Name: K ANSHU

Roll No: 2520030440

Section: 8

Skill: Quote Parsing, String Processing, Variable Expansion and Syntax Validation

Title

Implementation of Single and Double Quote Parsing with Variable Expansion.

Aim

To apply single quotes for preserving literal content, ignore variable expansion inside single quotes, store quoted strings, apply double quotes to preserve spaces while allowing variable expansion, parse quoted commands, and test edge cases.

Theory

Single quotes preserve the literal content inside them, so variable expansion is not performed. Double quotes preserve spaces within the quoted text while allowing environment variables to be expanded. The parser identifies quoted sections, stores the resulting tokens, validates the parsing result, and detects unclosed quotes.

Algorithm

1. Start the program.
2. Read command input from the user.
3. Check for the exit command and empty input.
4. Identify single-quoted strings and preserve their contents literally.
5. Identify double-quoted strings and preserve spaces inside them.
6. Expand environment variables found inside double quotes.
7. Store the resulting text as tokens.
8. Detect unclosed single or double quotes.
9. Display the parsed tokens and parsing result.
10. Repeat until exit is entered.
11. Stop the program.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 300
#define MAX_TOKENS 50
#define MAX_TOKEN 200

void add_char(char *token, int *len, char ch)
{
    if (*len < MAX_TOKEN - 1)
        token[(*len)++] = ch;
}

void expand_variable(const char *input, int *i, char *token, int *len)
```

```

{
    char name[100];
    int n = 0;

    (*i)++;

    while (input[*i] != '\0' &&
           (isalnum((unsigned char)input[*i]) || input[*i] == '_'))
    {
        if (n < 99)
            name[n++] = input[*i];

        (*i)++;
    }

    name[n] = '\0';

    char *value = getenv(name);

    if (value != NULL)
    {
        for (int j = 0; value[j] != '\0'; j++)
            add_char(token, len, value[j]);
    }

    (*i)--;
}

int parse_command(char *input)
{
    char tokens[MAX_TOKENS][MAX_TOKEN];
    int token_count = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        while (isspace((unsigned char)input[i]))
            i++;

        if (input[i] == '\0')
            break;

        int len = 0;

        while (input[i] != '\0' &&
               !isspace((unsigned char)input[i]))
        {
            if (input[i] == '\\')
            {
                i++;

                while (input[i] != '\0' && input[i] != '\\')
                {
                    add_char(tokens[token_count], &len, input[i]);
                    i++;
                }

                if (input[i] != '\\')
                {
                    printf("Syntax Error: Unclosed single quote.\n");
                    return 0;
                }

                i++;
            }
            else if (input[i] == '"')
            {
                i++;

                while (input[i] != '\0' && input[i] != '"')
                {

```

```

        if (input[i] == '$')
            expand_variable(input, &i,
                            tokens[token_count], &len);
        else
            add_char(tokens[token_count],
                    &len, input[i]);

        i++;
    }

    if (input[i] != '"')
    {
        printf("Syntax Error: Unclosed double quote.\n");
        return 0;
    }

    i++;
}
else
{
    add_char(tokens[token_count], &len, input[i]);
    i++;
}
}

tokens[token_count][len] = '\0';
token_count++;

if (token_count >= MAX_TOKENS)
    break;
}

printf("\nParsed Tokens:\n");

for (int j = 0; j < token_count; j++)
    printf("Token %d: [%s]\n", j + 1, tokens[j]);

printf("\nParsing successful.\n");

return 1;
}

int main()
{
    char input[MAX_INPUT];

    printf("=== OSSP Experiment 5 ===\n");
    printf("Single and Double Quote Parser\n");
    printf("Single quotes: literal content\n");
    printf("Double quotes: variable expansion allowed\n");
    printf("Type exit to quit.\n\n");

    while (1)
    {
        printf("quote> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
        {
            printf("Empty command.\n");
            continue;
        }
    }
}

```

```

        parse_command(input);
        printf("\n");
    }

    printf("Exiting quote parser...\n");

    return 0;
}

```

Execution Screenshots

1. Main C Code – Part 1

```

admin@heeraaj@DESKTOP-J39GHP7:~$
admin@heeraaj@DESKTOP-J39GHP7:~$ cat > main.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 300
#define MAX_TOKENS 50
#define MAX_TOKEN 200

void add_char(char *token, int *len, char ch)
{
    if (*len < MAX_TOKEN - 1)
        token[(*len)++] = ch;
}

void expand_variable(const char *input, int *i, char *token, int *len)
{
    char name[100];
    int n = 0;

    (*i)++;

    while (input[*i] != '\0' &&
           (isalnum((unsigned char)input[*i]) || input[*i] == '_'))
    {
        if (n < 99)
            name[n++] = input[*i];

        (*i)++;
    }

    name[n] = '\0';

    char *value = getenv(name);

    if (value != NULL)
    {
        for (int j = 0; value[j] != '\0'; j++)
            add_char(token, len, value[j]);
    }

    (*i)--;
}

int parse_command(char *input)
{
    char tokens[MAX_TOKENS][MAX_TOKEN];
    int token_count = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        while (isspace((unsigned char)input[i]))
            i++;

        if (input[i] == '\0')
            break;

        int len = 0;

        while (input[i] != '\0' &&
               !isspace((unsigned char)input[i]))

```

2. Main C Code – Part 2

```

        if (input[i] == '\\')
        {
            i++;

            while (input[i] != '\\0' && input[i] != '\\')
            {
                add_char(tokens[token_count], &len, input[i]);
                i++;
            }

            if (input[i] != '\\')
            {
                printf("Syntax Error: Unclosed single quote.\n");
                return 0;
            }

            i++;
        }
    else if (input[i] == '"')
    {
        i++;

        while (input[i] != '\\0' && input[i] != '"')
        {
            if (input[i] == '$')
                expand_variable(input, &i,
                                tokens[token_count], &len);
            else
            {
                add_char(tokens[token_count],
                        &len, input[i]);
            }

            i++;
        }

        if (input[i] != '"')
        {
            printf("Syntax Error: Unclosed double quote.\n");
            return 0;
        }

        i++;
    }
    else
    {
        add_char(tokens[token_count], &len, input[i]);
        i++;
    }
}

tokens[token_count][len] = '\\0';
token_count++;

if (token_count >= MAX_TOKENS)
    break;
}

printf("\nParsed Tokens:\n");

for (int j = 0; j < token_count; j++)
{
    printf("Token %d: [%s]\n", j + 1, tokens[j]);
}

```

3. Main C Code – Part 3 and Compilation

```

    printf("\nParsing successful.\n");
    return 1;
}

int main()
{
    char input[MAX_INPUT];

    printf("=== OSSP Experiment 5 ===\n");
    printf("Single and Double Quote Parser\n");
    printf("Single quotes: literal content\n");
    printf("Double quotes: variable expansion allowed\n");
    printf("Type exit to quit.\n\n");

    while (1)
    {
        printf("quote> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
        {
            printf("Empty command.\n");
            continue;
        }

        parse_command(input);
        printf("\n");
    }

    printf("Exiting quote parser...\n");
    return 0;
}
EOF

```

```

gcc main.c -o ossp
./ossp
=== OSSP Experiment 5 ===
Single and Double Quote Parser
Single quotes: literal content
Double quotes: variable expansion allowed
Type exit to quit.

```

```
quote> echo 'Hello $USER'
```

```

Parsed Tokens:
Token 1: [echo]
Token 2: [Hello $USER]

```

```
Parsing successful.
```

```
quote> Token 1: [echo]
```

```

Parsed Tokens:
Token 1: [Token]

```

4. Quote Parsing and Edge-Case Execution

```
Parsed Tokens:  
Token 1: [echo]  
Token 2: [Hello $USER]
```

```
Parsing successful.
```

```
quote> Token 1: [echo]
```

```
Parsed Tokens:  
Token 1: [Token]  
Token 2: [1:]  
Token 3: [[echo]]
```

```
Parsing successful.
```

```
quote> Token 2: [Hello $USER]
```

```
Parsed Tokens:  
Token 1: [Token]  
Token 2: [2:]  
Token 3: [[Hello]  
Token 4: [$USER]]
```

```
Parsing successful.
```

```
quote> echo "Hello $USER"
```

```
Parsed Tokens:  
Token 1: [echo]  
Token 2: [Hello admin dheeraj]
```

```
Parsing successful.
```

```
quote> echo "Hello World"
```

```
Parsed Tokens:  
Token 1: [echo]  
Token 2: [Hello World]
```

```
Parsing successful.
```

```
quote> Token 1: [echo]
```

```
Parsed Tokens:  
Token 2: [Hello Token 1: [Token]  
Token 2: [1:]  
Token 3: [[echo]]
```

Result

The single and double quote parser was executed successfully. Single-quoted content was preserved literally, double-quoted content allowed variable expansion and preserved spaces, and the parser detected an unclosed single quote.